

Modern szoftverfejlesztési eszközök

GKNB_INTM129

Specifikáció

CrabWar

Seamless-Stress csapata

Balaton Donát, Gyarmati Márk ,Sebők Roland Tamás

2026

Tartalomjegyzék

1. Alapinformációk	3
2. Történetvázlat	4
3. Követelmények	4
4. Használati esetek	6
5. Struktúra	6
5.1 Classok.....	7
5.2 Világ inicializása	7
6. Viselkedés	8
6.1 Aktivitás diagram leírás.....	10
6.2 Játék menete	10
7. Történet tárolási formátuma	10
7.1 Jaon file felépítése (példa)	10
8. Tesztelés	13
8.1 Test.....	13
8.2 Példák.....	13
9. Fejlesztés menete	14
9.1 Szoftverfejlesztési módszertanok	14
9.2 Dokumentáció.....	14
9.2.1 Doxygen	14
10. Ötletelés	15
9.2.1 Mindmap.....	15
11. Ötletelés	16
12 Fogalomtár	16

1. Alapinformációk

Ez a dokumentum egy 2D-s sci-fi kalandjáték specifikációja, amelynek a címe: CrabsOnCrob. A játékban szobáról szobára haladva rákok legyőzésével páncélok gyűjthetsz, amiket később NPC-knél pénzre válthatsz, amiből fegyvereket vásárolhatsz. A pálya felfedezése közben tárgyakat (item) szerezhetsz, amelyek segítik az előrehaladásodat.

2. Történetvázlat

A galaktikus naptár 15026. évében járunk, az emberiség meghódította a galaxist miután forradalmi faster than light (FTL) technológiát fejlesztettek ki, ami lehetővé tette a naprendszerek közötti utazást. Az FTL-utazáshoz speciális üzemanyag szükséges, amelynek az egyik alapanyaga a Crob-22 óceánbolygón élő rákok páncéljából készült őrlmény. A Crob-22 felszínét 89,8%-ban víz fedi. A rákvadászat egy veszélyes, de nagyon nyereséges munka, ami miatt sokan jönnek a bolygóra, de csak kevesen járnak sikerrel.

Főszereplőnk egy amatőr rákvadász szerepébe bújik, aki olyan céllal érkezik a Crob-22 nevű exoplanétára, hogy pénzt keressen szülei orvosi kezelésére, mivel a biztosító nem hajlandó kifizetni a szükséges egészségügyi ellátást. Szülei megmentése érdekében rövid idő alatt nagy zsákmányt kell szereznie. Miután főhősünk az utazási költségeket hiteltől biztosította, az elsődleges feladata ennek a tartozásnak a visszafizetése, csak ezt követően kezdheti meg a szülei részére szükséges pénz gyűjtését. Rövid időn belül nyilvánvalóvá válik számára, hogy az első cél elérése is emberpróbáló feladat, de már nincs visszaút, aláírta a hitelszerződést és csak akkor tud visszatérni a szüleihez, ha teljesíti a lehetetlent.

Te ekkor kapcsolódsz be a történetbe, a feladatod, hogy a Crob-22-n teljesítsd a lehetetlennek tűnő küldetést.

3. Követelmények

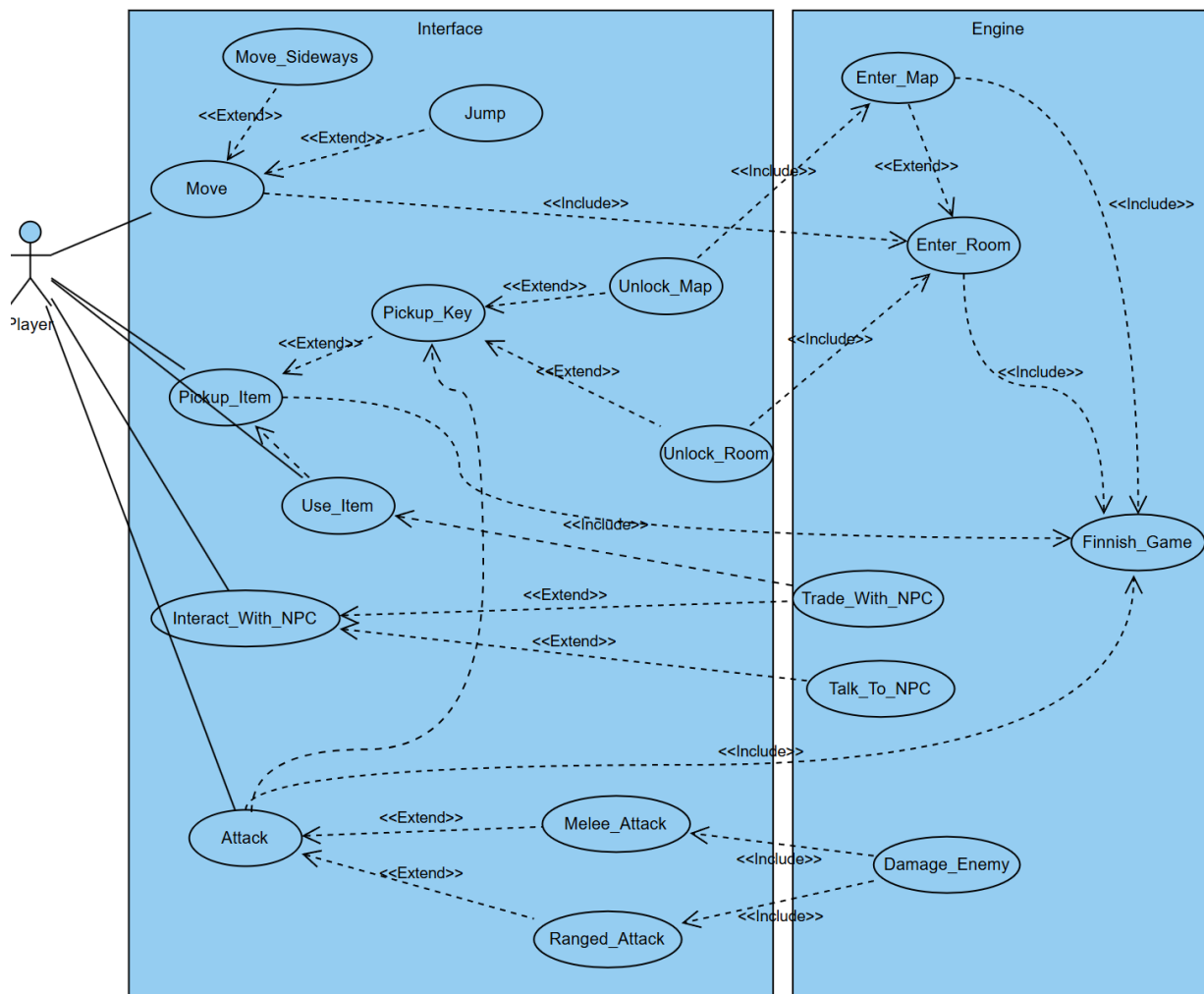
Táblázat 1: Követelmények

ID	Eset	Leírás	Prioritás
1	Szobarendszer	A játék szobáinak és a köztük lévő kapcsolatoknak a kialakítása.	Magas
2	Mozgás	A játékos legyen képes mozogni a szobákon belül és a szobák között is.	Magas
3	Támadás	A játékos legyen képes támadni az ellenségeknek.	Közepes
4	Történet	A történet beillesztése a játékmenetbe.	Közepes
5	Környezeti interakciók	A játékos tudjon kapcsolatba lépni a környezetével, tudjon szétszórt itemeket felvenni és használni.	Magas

4. Használati esetek

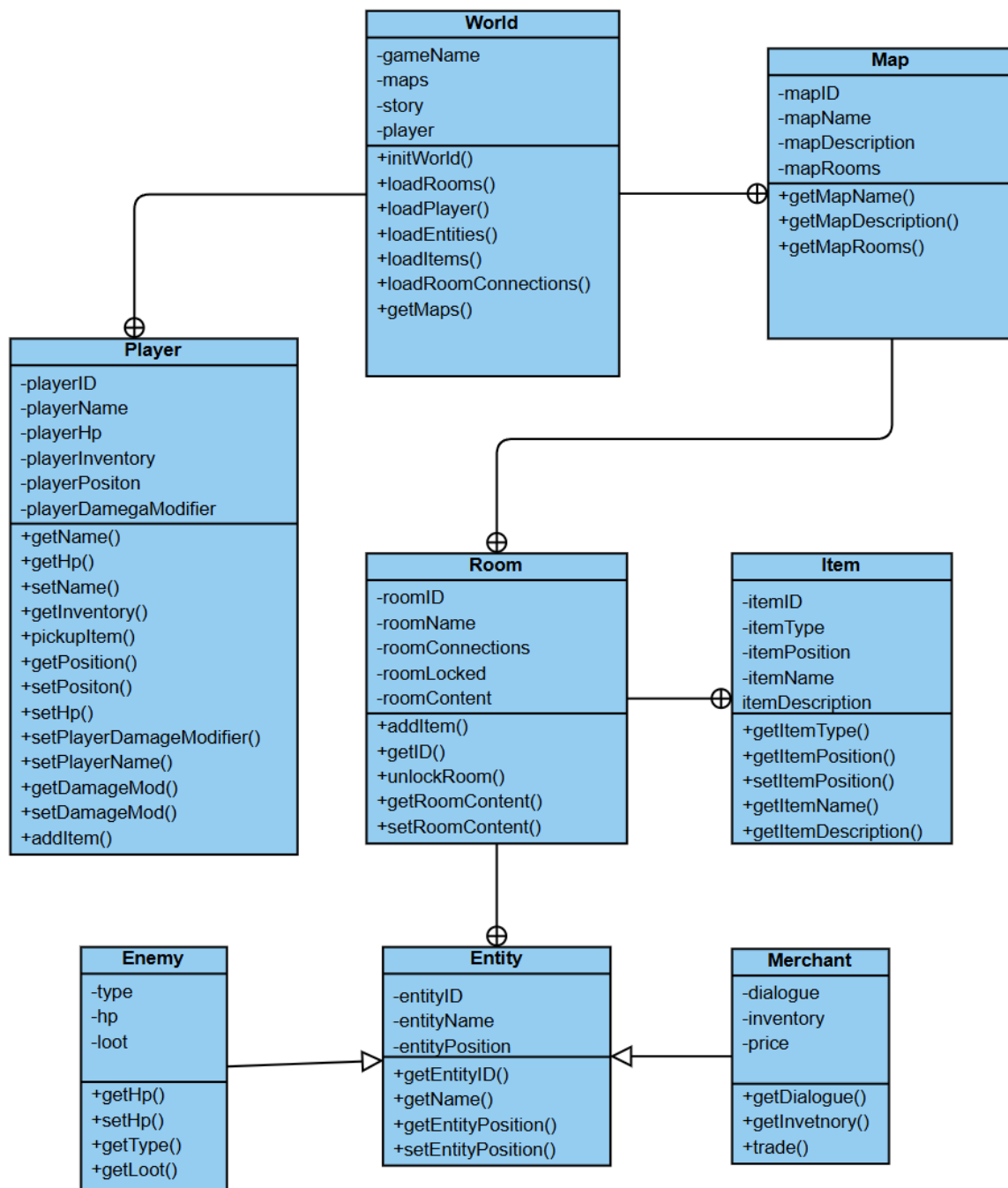
Táblázat 2: Követelmények

ID	Eset	Leírás	Prioritás
1.	Move	A játékos képes legyen balra, jobbra mozogni és ugrálni a pályán, valamint a szobák között mozogni.	Magas
2.	Attack	A játékos tudja támadni az enemy-eket. (közelről és távolról is)	Magas
3.	Pickup item	A játékos tudjon a pályán található szétszórt item-ek felvenni (Pl:lőszer, Healing item, „kulcs”)	Magas
4	Use item	Fogyó-item-ek használata (Pl:Hp, „kulcsok” ajtónyitásra)	Közepes
5	Interact with NPC	A pályán található NPC-kkel való beszélgetni és kereskedni.	Közepes
6	Unlock Map	Ha a játékos rendelkezik a megfelelő kulccsal, akkor nyíljon meg az új pálya.	Közepes
7	Finnish Game	Ha a játékos teljesítette az összes kritériumot a játék lezárásához, akkor érjen véget a játék, és jelenjen meg a lezáró jelenet	Magas



1. Ábra: Functional Use Cases

5. Struktúra



2. Ábra: Class Architecture

5.1 Classok

class World: A játék alapvető működéséért felelő class. Ez a class felel a világ inicializálásáért és a történethez fűződő beszélgetések, cutscene-ek megfelelő időben történő megjelenéséért.

class Player: Ez a class felel a játékos irányításáért és a játékoshoz köthető különböző interakciókért (pl: item felszedés).

class Map: A játék három jól elkülöníthető részből áll össze, ezeknek az elhatárolását segíti ez a class.

class Room: A játék világának a legalapvetőbb építőeleme a szoba, minden item és entity egy-egy ilyen szobához kapcsolódva van tárolva.

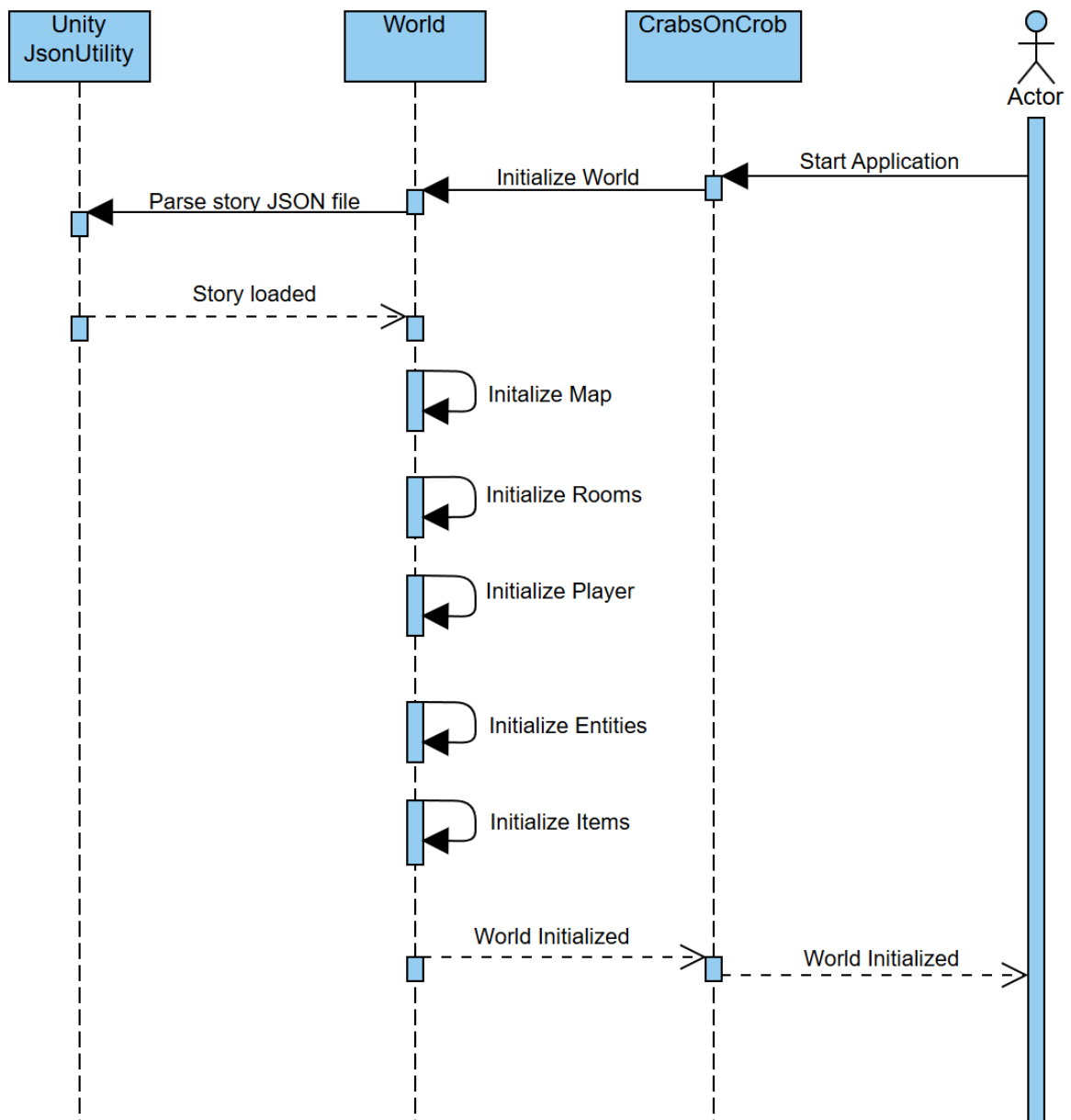
class Item: A játékos által felszedhető dolgok (item) pozíciójának és típusának a tárolására szolgáló class.

class Entity: A világ élőlényeiének tárolását segítő absztrakt class.

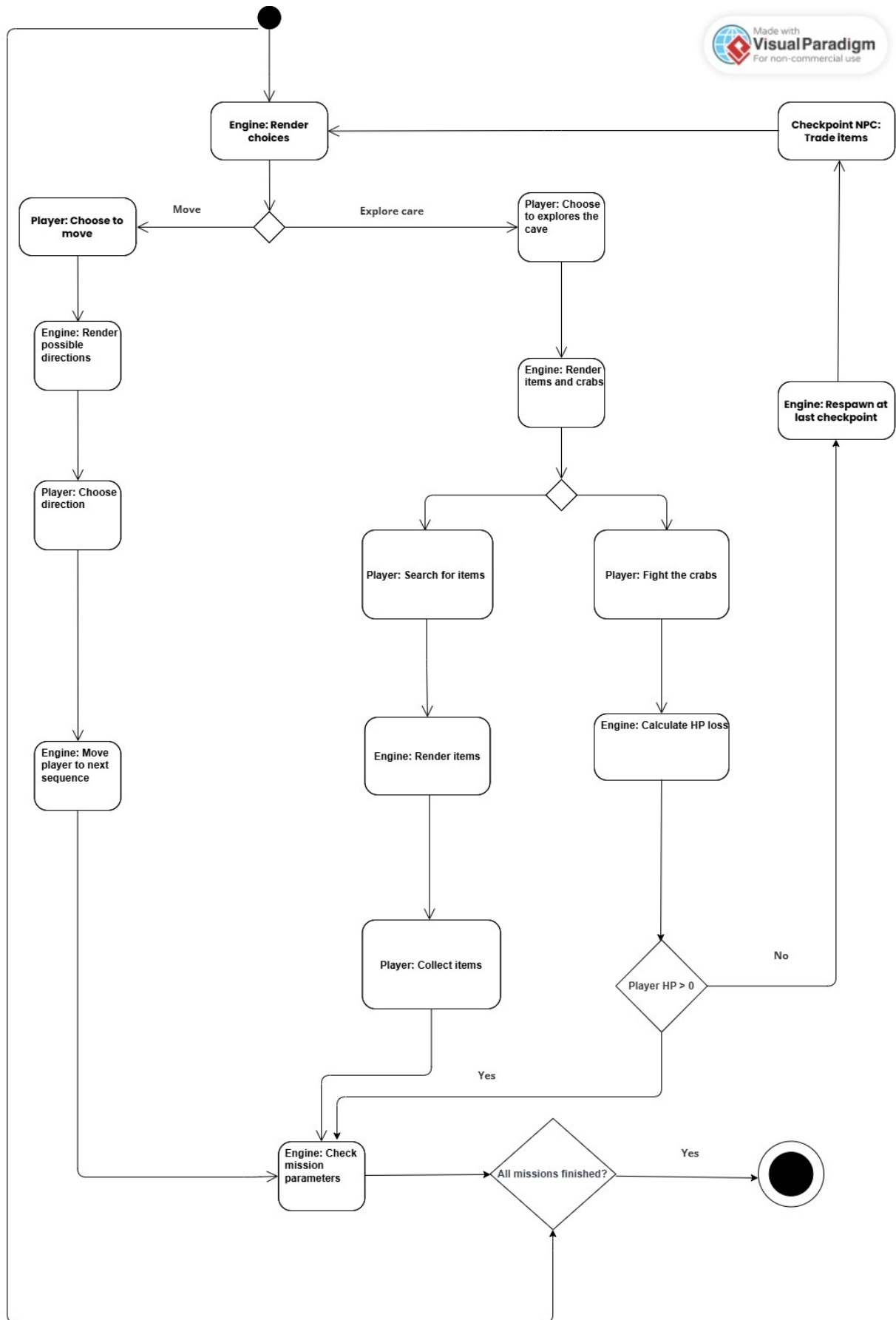
class Enemy: A játékban található mozgásra képes élőlényeket reprezentáló class, tartalmazza még az enemy-k halála során eső zsákmányt (loot-ot) is

class Merchant: A játék egyes pontjain lehetőség van árusokkal való beszélgetésre és kereskedésre, ezeknek a játékbeli élőlényeknek a reprezentációjára szolgál ez a class.

5.2 Világ inicializálás



6. Viselkedés



6.1 Aktivitás diagram leírás

A haladás során különböző tárgyakat gyűjthet össze és ezek pontokat adnak a játékos pontszámához. Bizonyos pontszintek elérésekor új események aktiválódhatnak, például új területek nyílhatnak meg vagy különleges tárgyak jelenhetnek meg. A pályán azonban veszélyek is találhatók, például csapdák vagy akadályok, amelyekbe a játékos beleeshet. Ilyenkor a játékos életerőt veszít (HP). Ha a játékos túl sok életerőt veszít és meghal, a játék az utolsó checkpointig dobja vissza. A checkpointnál található egy NPC is, akivel a játékos kereskedhet, például tárgyakat vásárolhat vagy felszerelést szerezhet a további haladáshoz.

6.2 Játék menete

A játék pályákra van osztva, egyik pálya kiválasztása után a játékos annak a pályának az elejére kerül. Ezután szabadon mozoghat, hogy felfedezzen, harcoljon enemy-kkel, itemeket gyűjtsön, NPC-kkel trade-eljen. Majd miután minden kulcsot megtalált a pályán odamehet a boss aréna ajtajához ami kinyílik és elkezdődik a bossfight. Miután pedig vége a bossfight-nak egy victory screen-en megjelennek statisztikák és visszaléphet a menübe vagy újrakezdheti a pályát.

7. Történet tárolási formátuma

A játék történetének tárolására json formátumot választottuk mivel ránézésre egyszerűen átlátható és megérthető és már alpból vannak rá eszközök a unity-ban a json fájlok kezelésére ezért nem kell semmilyen külső könyvtárat használni a történet tárolására.

7.1 Json file felépítésére egy példa

Itt egy AI által generált kicsi basic példával szemléltetni hogyan is néz ki egy json file és hogy hogyan használható.

Maga a json file:

```
{
  "startNode": "intro",
  "nodes": [
    {
      "id": "intro",
      "speaker": "oreg halasz",
      "text": "üdvözlét jóember segíthetek kalandra lelni?",
      "choices": [
        {
          "text": "Igen",
          "next": "adventure"
        },
        {
          "text": "nem csá",
          "next": "leave"
        }
      ]
    }
  ],
},
```

```
19 {
20   "id": "adventure",
21   "speaker": "oreg halasz",
22   "text": "Akkor vigyed ezt a szigonyt és menyé rákokat ölni.",
23   "choices": [
24     {
25       "text": "Köszönöm",
26       "next": "end"
27     }
28   ]
29 },
30 {
31   "id": "leave",
32   "speaker": "oreg halasz",
33   "text": "Ejjj a mai fiatalság. ",
34   "choices": []
35 }
36 ]
37 }
```

A fájl dialógus node-okból áll amiken belül ki lehet írni magát a dialógust és válsztási lehetőségeket kínálni.

C#-ban a json file-nak a struktúrája:

```

[System.Serializable]
public class DialogueChoice
{
    public string text;
    public string next;
}

[System.Serializable]
public class DialogueNode
{
    public string id;
    public string speaker;
    public string text;
    public DialogueChoice[] choices;
}

[System.Serializable]
public class DialogueData
{
    public string startNode;
    public DialogueNode[] nodes;
}

```

És a dialógus betöltése:

```

using UnityEngine;

public class DialogueLoader : MonoBehaviour
{
    public TextAsset dialogueFile;
    private DialogueData dialogue;

    void Start()
    {
        dialogue = JsonUtility.FromJson<DialogueData>(dialogueFile.text);
        StartDialogue();
    }

    void StartDialogue()
    {
        ShowNode(dialogue.startNode);
    }

    void ShowNode(string nodeId)
    {
        foreach (DialogueNode node in dialogue.nodes)
        {
            if (node.id == nodeId)
            {
                Debug.Log(node.speaker + ": " + node.text);

                foreach (DialogueChoice choice in node.choices)
                {
                    Debug.Log("Choice: " + choice.text);
                }

                break;
            }
        }
    }
}

```

8. Tesztelés

8.1 Tesztelés

A teszteléshez a Unity beépített “test framework” csomagját fogjuk használni, mivel így Unity-n belül tudjuk kezelni a tesztelést is. Néhány kattintással az engine legenerál nekünk néhány sablon tesztet és a teszt struktúrát amit utána átalakítható igény szerint.

8.2 Példa

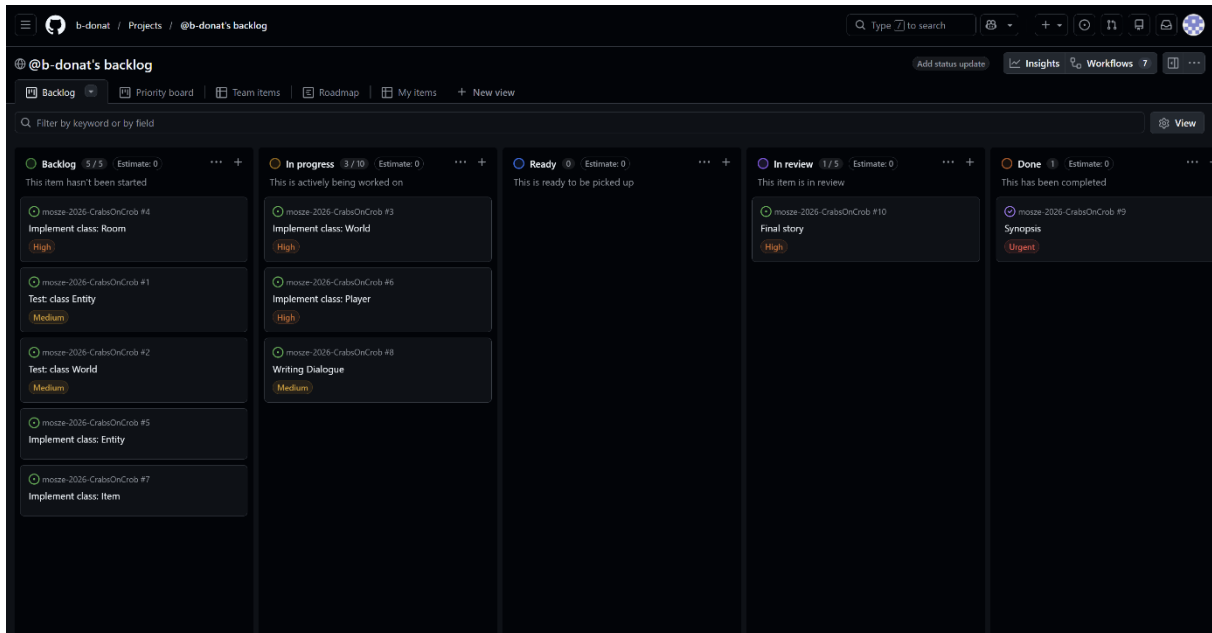
Sablon tesztet amit a Unity generál

```
1  using System.Collections;
2  using NUnit.Framework;
3  using UnityEngine;
4  using UnityEngine.TestTools;
5
6  0 references
7  public class NewTestScript
8  {
9      // A Test behaves as an ordinary method
10     [Test]
11     0 references
12     public void NewTestScriptSimplePasses()
13     {
14         // Use the Assert class to test conditions
15     }
16
17     // A UnityTest behaves like a coroutine in Play Mode. In Edit Mode you can use
18     // `yield return null;` to skip a frame.
19     [UnityTest]
20     0 references
21     public IEnumerator NewTestScriptWithEnumeratorPasses()
22     {
23         // Use the Assert class to test conditions.
24         // Use yield to skip a frame.
25         yield return null;
26     }
27 }
```

9. Fejlesztés menete

9.1 Szoftverfejlesztési módszertanok

A játék fejlesztése során a scrum módszertant tervezünk követni, a gyors iteratív fejlesztés érdekében. A backlogok nyomonkövetésére és az átláthatóság javítására, a GitHubon belüli „Projects” fülön található kanban táblát használjuk.



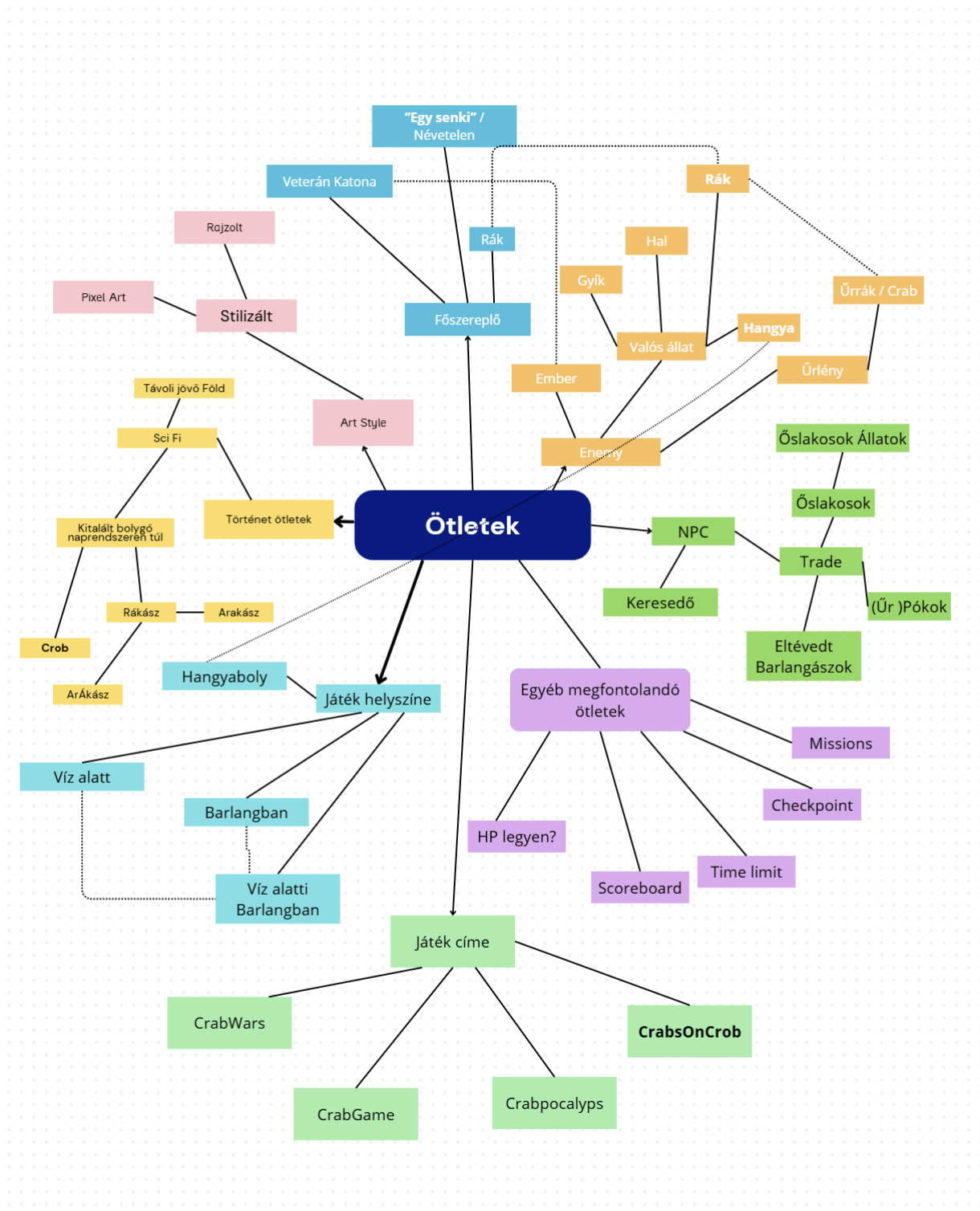
9.2 Dokumentáció

9.2.1 Doxygen

A fejlesztés során Doxygen-t tervezünk használni, mivel átláthatóbbá teszi a kódot és segít a csapatmunkát.

10. Ötletelés

10.1 Mindmap



11. Fejlesztési eszközök

- Verziókezelés: Git, Szolgáltató Github
- Fejlesztői környezet: Visual Studio Code
- Játékmotor: Unity
- Diagramok elkészítéséhez használt eszközök: VisualParadigm (Online), PlantUML, Modelio
- Operációs rendszerek: Windows 10, Windows 11
- Dokumentáláshoz használt eszközök: Markdown, Doxygen, LaTeX,
- Tesztelés: Unity (test framework)
- Debugger eszközök: Visual Studio Debugger, *Unity Profiler*
- Szövegszerkesztő: Microsoft Word, Google Dokumentumok
- Kommunikációs platformok: Messenger, Microsoft Teams
- Grafikai készítő: Canva

12. Fogalomtár

- Kalandjáték: Egy olyan videójáték műfaj, amely a felfedezést, döntéseket és a problémamegoldást helyezi előtérbe.
- Specifikáció: Egy rendszer vagy program részletes leírása, amely felvázolja annak működését és jellemzőit.
- Sci-Fi (*science fiction*): Olyan művészeti alkotás, mely legtöbbször valódi vagy képzeletbeli tudomány(ok)-nak a társadalomra, vagy egyes egyénekre gyakorolt hatását mutatja be.
- FTL (Faster Than Light) technológia: Elméleti űrutazási technológia, amely lehetővé teszi az űrhajók számára, hogy a fénysebességnél gyorsabban haladjanak, így rövid idő alatt hatalmas távolságokat tudnak megtenni a galaxisban.
- Galaktikus év: A történet világában használt naptárrendszer, amely az emberiség galaktikus korszakához kapcsolódik.
- Cutscene: Videójáték esetében egy olyan átvezetőjelenet, ami a történetmesélését viszi előre, nem interaktív azaz a játékosnak nincsen közvetlen hatása rá.
- HP (Healt Points): A videójátékokban gyakran használt érték, mely a játékosok és egyéb entity-k életerejét számszerűsíti
- NPC (Non-Playabel-Character): A játékban szereplő olyan karalterek, amelyek a játékos által nem irányíthatóak.

- Enemy: Egy ellenséges karakter egy videojátékokban.
- Item: Olyan videojátékokban található tárgy ami általában felszedhető és valamilyen funkciót lát el (pl: gyógyít).
- Tesztelés: Egy rendszer vagy program kontrollált körülmények melletti futtatása, és az eredmények kiértékelése. A hagyományos megközelítés szerint a tesztelés célja az, hogy a fejlesztés során létrejövő hibákat minél korábban felfedezze, és ezzel csökkentse azok kijavításának költségeit.
- Scrum: A szoftverfejlesztés területén gyakran használt agilis szoftverfejlesztés egyik fontos összetevője.
- Kanban: Emberek csoportos munkájának menedzselésére és fejlesztésére kialakított
- ütemezési rendszer.
- Git: Gráfolapú nyílt forráskódú protokoll verziókezelésre
- Github: A Microsoft által üzemeltetett felhőalapú git protokollt használó verziókezelő tárhely.
- Visual Studio Code: Egy ingyenesen elérhető, könnyű kódszerkesztő.
- Unity: Az egyik legelterjedtebb videójáték-motor.
- Microsoft Word: Egy szövegszerkesztő alkalmazás
- Google Dokumentumok: A Google felhőalapú szövegszerkesztője, megkönnyíti a közös dokumentumok szerkesztését.
- Messenger: Egy azonnali üzenetküldő alkalmazás, ami a Meta tulajdonában van.
- Microsoft Teams: Egy a Microsoft által fejlesztett kommunikációs és együttműködési platform,
- Canva: A Canva egy könnyen használható, online grafikai tervezőplatform.